CITIZEN-TO-GOVERNMENT CIVIC TECH PLATFORM

Sewage Alert

Hyderabad

A cloud-native microservices platform that lets citizens report sewage and sanitation issues with photo + GPS, and lets authorities assign, track and resolve them with proof.

v1.1.0 · Technical Presentation · August 2026

THE PROBLEM WE SET OUT TO SOLVE

Sewage issues were invisible and slow to fix

Citizens had no reliable channel to report sanitation problems and no visibility into resolution.

No easy channel

Complaints lost across phone calls, social media and word of mouth — no single source of truth.

No transparency

Citizens could not track whether — or when — their issue would be fixed.

No accountability

Authorities lacked a structured queue, assignment and proof of work.

WHAT WE BUILT

One structured platform

Report with photo + GPS in under a minute.

Real-time tracking

Every status change is notified, stored and visible.

Proof-based resolution

v1.1.0 mandates a resolution photo before closing (enforced at the backend).

WHAT THE PLATFORM DOES

Key features

Built for four roles — Citizen, Authority/Admin, Field Officer and the general public.

Report with photo & GPS

Compressed JPG uploads to Cloudinary; only URLs stored.

Hotspot heatmap

Admin analytics over complaint density, status and priority.

Assign to field officers

Admin assigns; officers see only their own queue.

Resolution proof (v1.1.0)

Mandatory proof photo before a complaint is RESOLVED.

Event-driven notifications

RabbitMQ → in-app notifications + EmailJS emails.

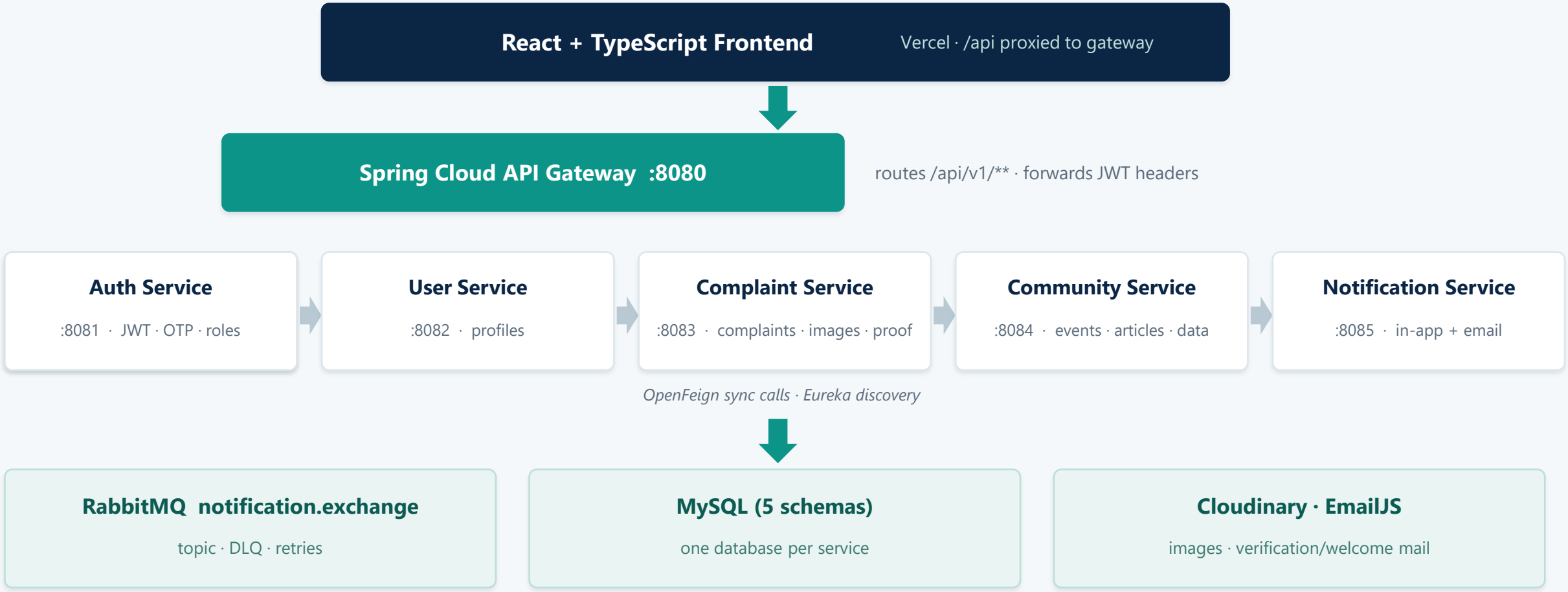
Community & awareness

Events, articles, NGOs, lakes, STPs & pipelines.

HOW IT FITS TOGETHER

System architecture

Microservices behind a single API Gateway, with per-service databases and an event bus.



Microservice boundaries are enforced with per-service databases and OpenFeign inter-service calls — no shared schema, no hidden coupling.

WHAT IT IS BUILT WITH

Technology stack

Proven, mainstream technologies chosen for team familiarity and operational simplicity.

Backend

- **Java 25 · Spring Boot**
  microservices runtime
- **Spring Security + JWT**
  stateless auth, BCrypt
- **Spring Data JPA / Hibernate**
  persistence, ddl-auto:update
- **Spring Cloud Gateway + Eureka**
  routing & discovery
- **OpenFeign**
  sync service-to-service calls
- **RabbitMQ (Spring AMQP)**
  async event bus + DLQ
- **MySQL 8 / H2**
  per-service databases

Frontend

- **React 18 + TypeScript**
  typed UI components
- **Vite**
  fast dev + builds
- **Tailwind CSS**
  design-token styling
- **React Router 6**
  role-based routes
- **Recharts**
  admin analytics
- **Leaflet + react-leaflet**
  hotspot heatmaps
- **Lucide icons**
  consistent iconography

DevOps & Integrations

- **Docker + Docker Compose**
  7 services + MySQL + RabbitMQ
- **GitHub Actions**
  backend/frontend CI + image push
- **Docker Hub**
  image registry
- **AWS EC2**
  backend hosting
- **Vercel**
  frontend hosting (SPA)
- **Cloudinary**
  object storage for images
- **EmailJS**
  verification & welcome email

SEVEN DEPLOYABLE UNITS, ONE PRODUCT

Microservices at a glance

Each service owns its domain, its database and its public contract.

Auth Service	**:8081**	`sewagealert_auth`	Register, login, JWT issuance, OTP email verification, role lookup.
User Service	**:8082**	`sewagealert_users`	Citizen profile CRUD; consumed via OpenFeign by other services.
Complaint Service	**:8083**	`sewagealert_complaints`	Complaint lifecycle, Cloudinary images, assignment, status, v1.1.0 resolution proof.
Community Service	**:8084**	`sewagealert_community`	Events + registrations, articles, NGOs, lakes, STPs, pipelines; external APIs (GNews, Google Places, Overpass, ArcGIS).
Notification Service	**:8085**	`sewagealert_notifications`	RabbitMQ consumer → in-app notifications + EmailJS emails; DLQ observer.
Eureka Server	**:8761**	–	Service discovery registry for the gateway and Feign clients.
API Gateway	**:8080**	–	Single entry point; routes /api/v1/**; forwards X-Auth-User-Id + JWT.

IDENTITY, TRUST AND AUTHORIZATION

Authentication & security flow

Email-verified accounts, stateless JWT, and server-side role checks on every protected action.

1 · Register

1

BCrypt password hash; account created UNVERIFIED.

2 · Verify email

2

6-digit OTP via EmailJS; max 5 attempts + 60s lockout.

3 · Login

3

JWT issued: sub, email, role, iat, exp (24h).

4 · Call APIs

4

Authorization: Bearer <jwt> + X-Auth-User-Id header.

JWT payload (claims actually used)

`sub`	user id (auth-service)
`email`	user email
`role`	CITIZEN · AUTHORITY · ADMIN · FIELD_OFFICER
`iat / exp`	issued-at / 24-hour expiry

Security controls enforced in code

- ✓ BCryptPasswordEncoder for all stored passwords
- ✓ Stateless sessions — JWT validated by JwtAuthenticationFilter
- ✓ Gateway injects X-Auth-User-Id; services re-verify roles via AUTH-SERVICE
- ✓ Public only: register / login / verify-code / resend / swagger
- ✓ Unauthorized or expired token → 401 → frontend signs out

THE CORE WORKFLOW — AND THE V1.1.0 HEADLINE FEATURE

Complaint lifecycle

Citizen reports → authority acts → field officer fixes → proof uploaded → resolved.

PENDING

Citizen submits with photo + GPS

→

IN_PROGRESS

Authority picks up / assigns officer

→

RESOLVED

Officer uploads mandatory proof photo

NOTIFIED — RabbitMQ event → in-app notification + email, at every status transition

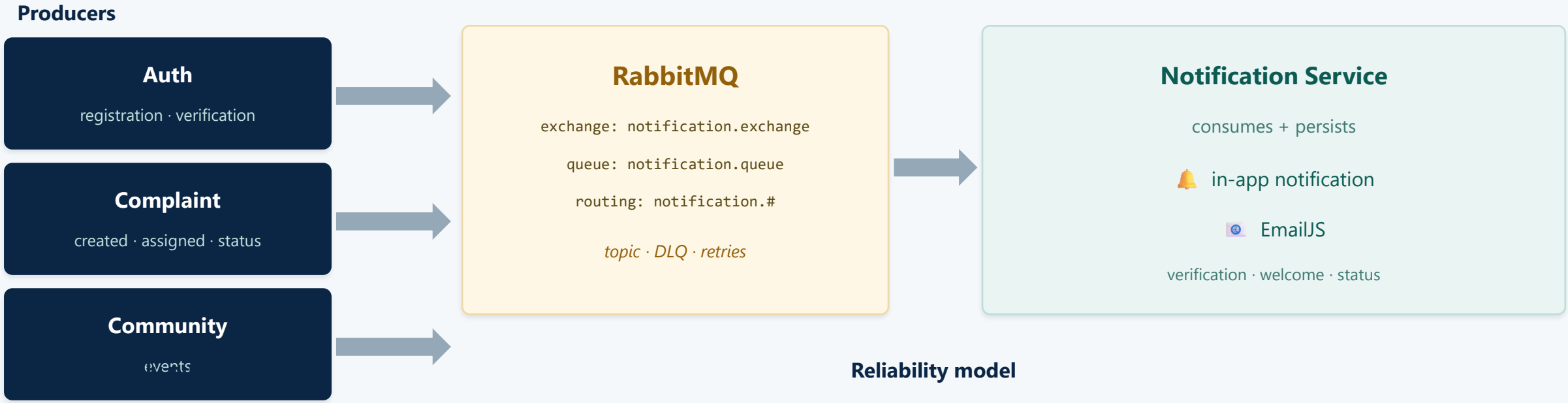
✨ v1.1.0 — Mandatory resolution proof photo (backend-enforced)

Backend rule	JSON status PATCH rejects RESOLVED; only POST …/resolve (multipart) can resolve.
Upload-first	Photo validated (JPG/PNG/WEBP, ≤10 MB) and uploaded to Cloudinary BEFORE the status change.
Safe failure	A failed upload leaves the complaint untouched — never silently resolved.
Transparency	Proof URL (resolutionProofImageUrl) is returned and shown to the citizen.

ASYNC BY DESIGN

Event-driven notifications

Synchronous REST for requests; RabbitMQ for reliable, decoupled event delivery.



- `COMPLAINT_CREATED → complaint → citizen`
- `COMPLAINT_ASSIGNED → → field officer`
- `COMPLAINT_STATUS_UPDATED / RESOLVED / REJECTED / REOPENED → citizen`
- `USER_REGISTERED / EMAIL_VERIFICATION_REQUESTED / EMAIL_VERIFIED → new citizen`
- `COMMUNITY_EVENT → citizens (future-ready)`

Reliability model

- ✓ Durable queue; JSON via Jackson converter (INFERRED type mapping)
- ✓ Container retry: exponential backoff, max 5 attempts
- ✓ Permanent failures → dead-letter exchange → notification.dlq
- ✓ DLQ consumer logs every parked message for ops/replay
- ✓ Publishing is fire-and-forget — a broker outage never blocks a request

THREE EXPERIENCES, ONE CODEBASE

Frontend — React 18 + TypeScript

Role-based routing with route guards; JWT in localStorage; all traffic through the gateway.

Public	/ · /login · /register · /track · /events · /articles · /ngos · /lakes · /infrastructure
Citizen	/dashboard — report with photo+GPS, my complaints, detail + proof, notifications, profile
Admin / Authority	/admin — command centre, complaints (assign / status / resolve-with-proof), analytics, heatmap, community CRUD, settings
Field Officer	/officer — assigned complaints, status updates, resolve with mandatory proof photo

Client-side image compression (object URLs, no base64 in state) · toasts · empty states · SPA fallback for deep links

FROM COMMIT TO PRODUCTION

Docker, CI/CD & deployment

GitHub Actions builds and tests; Docker Hub hosts images; AWS + Vercel run them.

Pipeline

Developer push
→ PR to develop/main

Backend CI
mvn clean verify (JDK 25)

Frontend CI
npm ci → build

Docker build+push
7 images → Docker Hub

Production
AWS EC2 + Vercel

Local development (docker compose)

- ✓ docker compose up -d --build — 9 containers on one bridge network
- ✓ MySQL :3307 · RabbitMQ :5672 + :15672 · Eureka :8761
- ✓ Services :8080–8085; frontend dev on :5173 (proxy /api → :8080)
- ✓ Health checks gate service startup (mysqladmin, rabbitmq-diagnostics)

Production topology

- ✓ docker-compose.prod.yml pulls images from Docker Hub (no builds on host)
- ✓ Backend on AWS EC2; frontend static SPA on Vercel with /api rewrite
- ✓ Secrets via deployment .env — never committed (GitHub secrets for CI)
- ✓ v1.1.0 features ship the same way: CI → image → compose pull → restart

WHERE WE ARE AND WHERE WE ARE GOING

Roadmap

v1.0	*Released*	Core platform: reporting, tracking, notifications, auth, Docker, CI/CD
v1.1	*Now*	Resolution proof photos (backend-enforced) + user-facing content cleanup
v1.2	*Next*	Responsive/mobile-friendly web experience
v2.0	*Planned*	NGO & community engagement platform (events, drives, approvals)
v2.1	*Planned*	AI capabilities on top of structured platform data
v3.0	*Planned*	Dedicated mobile app reusing the existing backend APIs

Thank you

Questions welcome — siddh@sewagealert.hyderabad